

# ONBRD-06: Organizing Your Environment

---

**Series:** ONBRD — Dynatrace Onboarding | **Notebook:** 6 of 10 | **Created:** December 2025 | **Last Updated:** 09/18/2026

## Tags, Segments, and Naming Conventions

---

As your Dynatrace environment grows, organization becomes critical. This notebook covers how to structure your environment with tags, segments, and naming conventions for maintainability and access control.

---

## Table of Contents

---

1. [Why Organization Matters](#)
  2. [Modern Organization Building Blocks](#)
  3. [Tagging with Host Properties and Cloud Tags](#)
  4. [Segments for Data Filtering](#)
  5. [Naming Conventions](#)
  6. [Querying by Tags and Properties](#)
  7. [Next Steps](#)
- 

## Prerequisites

---

- Admin or Configurator access to Dynatrace
- Entities discovered (hosts, services)
- Understanding of your organizational structure

# OneAgent Attribute Enrichment (OneAgent 1.333+)

**Requires:** OneAgent version **1.333** or later

OneAgent can enrich **all telemetry signals** (metrics, spans, logs, events, entities) with custom metadata at the source — before data reaches the Dynatrace platform. This is more efficient than server-side tagging (auto-tags) because enrichment happens on the host and propagates to all Smartscape nodes.

**Primary Fields** (standardized from Semantic Dictionary):

- `dt.security_context` — data governance and access control
- `dt.cost.costcenter` — cost allocation
- `dt.cost.product` — product attribution

**Primary Tags** (custom key-value pairs):

- `primary_tags.environment` — environment identification (production, staging, etc.)
- `primary_tags.team` — team ownership
- `primary_tags.business_unit` — organizational unit

**Configuration:**

```
# Set during OneAgent installation
Dynatrace-OneAgent-Linux.sh --set-host-tag="primary_tags.environment=production" --set-host-tag="dt.security_context=confidential"

# Set on existing agents via oneagentctl
oneagentctl --set-host-tag="primary_tags.environment=production"
oneagentctl --set-host-tag="dt.cost.costcenter=12345"

# Per-process via environment variable (overrides host-level)
DT_TAGS="primary_tags.team=platform primary_tags.environment=production"
```

**Benefits over auto-tagging:**

|                   |                                |                                                                   |
|-------------------|--------------------------------|-------------------------------------------------------------------|
| Aspect            | Auto-Tags (Server-Side)        | Attribute Enrichment (Agent-Side)                                 |
| -----             | -----                          | -----                                                             |
| When applied      | After data arrives at platform | At the source, before transmission                                |
| Scope             | Entity tags only               | All signals: metrics, spans, logs, events, entities               |
| Grail integration | Limited                        | Full — feeds OpenPipeline routing, bucket assignment, permissions |

| Cost allocation | Not supported | `dt.cost.costcenter` , `dt.cost.product` fields |  
| Security context | Not supported | `dt.security_context` for data governance |

See: [Primary Grail fields and tags enrichment through OneAgent](#)

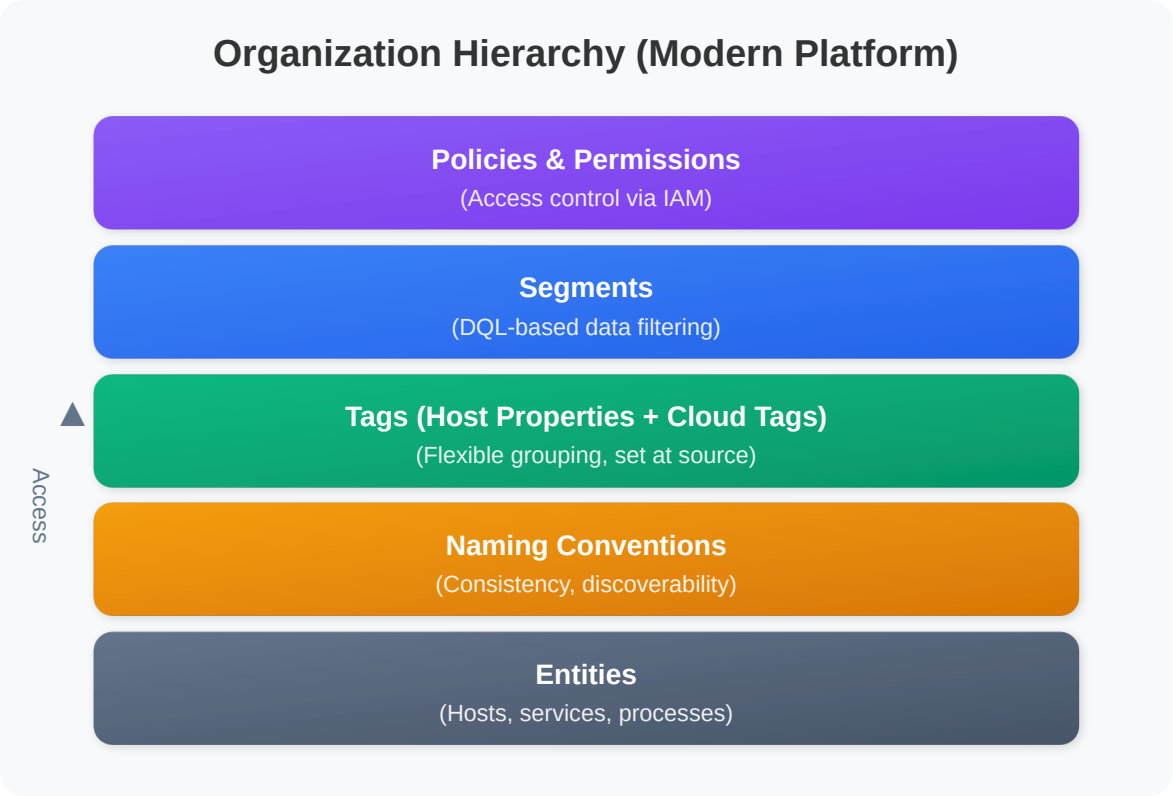
**Version Support Policy:** OneAgent and ActiveGate versions are supported for **9 months (Standard)** or **12 months (Enterprise)**. Third-party technologies are supported for 6 months beyond vendor EOL. See [Support Policy](#).

## 1. Why Organization Matters

Without organization, Dynatrace environments become difficult to manage:

| Problem           | Impact                          |
|-------------------|---------------------------------|
| No structure      | Can't find entities quickly     |
| No ownership      | Don't know who to contact       |
| No access control | Everyone sees everything        |
| No filtering      | Dashboards show irrelevant data |
| No grouping       | Can't compare similar systems   |

# Modern Organization Building Blocks



**Note:** The modern Dynatrace platform uses **Segments** for data filtering and **Policies** for access control. This replaces the legacy Management Zones approach.

## 2. Modern Organization Building Blocks

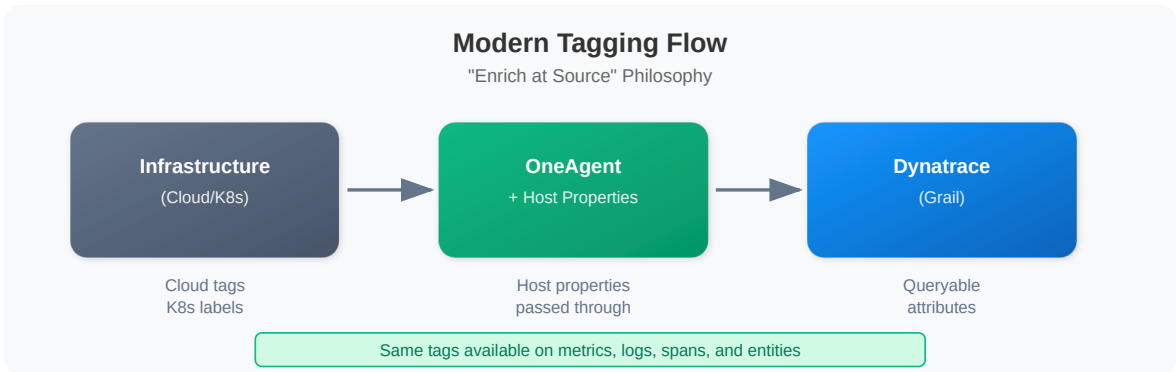
The modern Dynatrace platform (Gen3/Grail) uses a "tag at source" approach rather than rule-based auto-tagging:

### Tag Sources

| Source              | How It Works                              | Best For                         |
|---------------------|-------------------------------------------|----------------------------------|
| Host Properties     | Set via OneAgent configuration            | Environment, team, tier metadata |
| Cloud Provider Tags | Automatically imported from AWS/Azure/GCP | Cloud resource organization      |

| Source                   | How It Works               | Best For                        |
|--------------------------|----------------------------|---------------------------------|
| Kubernetes Labels        | Imported from K8s metadata | Container workload organization |
| OpenTelemetry Attributes | Set in instrumentation     | Custom service attributes       |

## The "Enrich at Source" Philosophy



## Why "Tag at Source"?

| Benefit    | Description                                     |
|------------|-------------------------------------------------|
| Consistent | Same tags on metrics, logs, spans, and entities |
| Scalable   | No processing overhead to apply rules           |
| Traceable  | Tags come from the source of truth              |
| Real-time  | No delay waiting for rule evaluation            |

## 3. Tagging with Host Properties and Cloud Tags

### Host Properties (OneAgent)

Set custom properties during OneAgent installation or via configuration:

**During Installation:**

```
# Linux
sudo /bin/sh Dynatrace-OneAgent.sh \
  --set-host-property=env=production \
  --set-host-property=team=platform \
  --set-host-property=tier=backend

# Windows
.\Dynatrace-OneAgent.exe --set-host-property=env=production --set-host-property=team=checkout
```

### Via Configuration File:

**Note:** The configuration file path depends on your OneAgent version. Check your version with `oneagentctl --version` to determine the correct path.

| OneAgent Version | Configuration File Path                                                         |
|------------------|---------------------------------------------------------------------------------|
| < 1.225          | <code>/var/lib/dynatrace/oneagent/agent/config/hostcustomproperties.conf</code> |
| ≥ 1.225          | <code>/var/lib/dynatrace/oneagent/agent/config/custom.properties</code>         |

```
# OneAgent < 1.225:
/var/lib/dynatrace/oneagent/agent/config/hostcustomproperties.conf
# OneAgent ≥ 1.225: /var/lib/dynatrace/oneagent/agent/config/custom.properties
env=production
team=platform
cost-center=engineering
```

### Recommended Property Categories

| Category    | Example Properties                                                      | Purpose                   |
|-------------|-------------------------------------------------------------------------|---------------------------|
| Environment | <code>env=prod</code> , <code>env=staging</code> , <code>env=dev</code> | Distinguish environments  |
| Owner       | <code>team=platform</code> , <code>team=checkout</code>                 | Identify responsible team |
| Application | <code>app=ecommerce</code> , <code>app=mobile-api</code>                | Group by application      |
| Cost Center | <code>cost-center=marketing</code>                                      | Financial allocation      |
| Tier        | <code>tier=frontend</code> , <code>tier=backend</code>                  | Architecture layer        |

## Cloud Provider Tags

Cloud tags are automatically imported when using cloud integrations:

| Cloud | Tag Format          | DQL Field                |
|-------|---------------------|--------------------------|
| AWS   | AWS resource tags   | <code>aws.tag.*</code>   |
| Azure | Azure resource tags | <code>azure.tag.*</code> |
| GCP   | GCP labels          | <code>gcp.label.*</code> |

### AWS Tag Example:

If your EC2 instance has tag `Environment=Production`, it appears as `aws.tag.Environment` in Dynatrace.

## Kubernetes Labels

K8s labels are automatically available for container workloads:

| K8s Metadata | DQL Field                        |
|--------------|----------------------------------|
| Namespace    | <code>k8s.namespace.name</code>  |
| Deployment   | <code>k8s.deployment.name</code> |
| Pod labels   | <code>k8s.pod.labels.*</code>    |
| Node labels  | <code>k8s.node.labels.*</code>   |

## 4. Segments for Data Filtering

Segments provide DQL-based filtering to create focused views of your data.

**Location:** Observe and explore → Segments

### What are Segments?

Segments are reusable DQL filters that:

- Filter data in Notebooks, Dashboards, and Apps
- Can be applied as default context
- Are shareable across the organization

## Creating a Segment

1. Go to Observe and explore → Segments
2. Click "Create segment"
3. Define your DQL filter (using current Smartscape/property syntax — `dt.entity.*` is deprecated):  

```
dql properties.env == "production"
```
4. Name your segment (e.g., "Production Environment")
5. Save

## Segment Use Cases

| Use Case    | Segment Filter                                 | Purpose              |
|-------------|------------------------------------------------|----------------------|
| Environment | <code>properties.env == "prod"</code>          | Focus on production  |
| Team        | <code>properties.team == "checkout"</code>     | Team-specific view   |
| Application | <code>contains(service.name, "payment")</code> | Application focus    |
| Region      | <code>aws.tag.Region == "us-east-1"</code>     | Geographic filtering |

**DQL syntax note:** `contains` is a function call — `contains(service.name, "payment")` — not a SQL-style infix operator. The pattern `service.name contains "payment"` is invalid DQL.

## Segments vs Legacy Management Zones

| Feature        | Segments                                                | Management Zones (Legacy) |
|----------------|---------------------------------------------------------|---------------------------|
| Filter basis   | DQL expressions                                         | Rule-based matching       |
| Data types     | All Grail data                                          | Entities only             |
| Flexibility    | Highly flexible                                         | Limited rule types        |
| Access control | Use Policies + <code>dt.security_context</code> instead | Built-in                  |

| Feature         | Segments      | Management Zones (Legacy)                                                          |
|-----------------|---------------|------------------------------------------------------------------------------------|
| Modern platform | ✅ Recommended | ⚠️ Being phased out — MZ-on-calculated-metrics is on the May-2026 deprecation list |

**Where to go deeper:** the **ORGNZ series** (11 notebooks) covers segments, buckets, and `dt.security_context` design in depth. The **IAM series** (especially IAM-04, IAM-05, IAM-11 WORKSHOP) covers how policies use `dt.security_context` to scope access. **MZ2POL** (11 notebooks) covers Management Zone → Policy migration if you have legacy MZs to retire.

## Segment Best Practices

| Practice                                               | Why                                                       |
|--------------------------------------------------------|-----------------------------------------------------------|
| Use host properties / <code>dt.security_context</code> | Consistent filtering; aligns with IAM scoping             |
| Name clearly                                           | <code>Prod-Checkout-Team</code> not <code>Segment1</code> |
| Document purpose                                       | Add description                                           |
| Test filters                                           | Verify expected data                                      |

## 5. Naming Conventions

Consistent naming makes entities discoverable and filtering effective.

### Host Naming

Set meaningful host names that encode key information:

| Pattern                            | Example                         | Components                  |
|------------------------------------|---------------------------------|-----------------------------|
| <code>{env}-{tier}-{seq}</code>    | <code>prod-web-01</code>        | Environment, tier, sequence |
| <code>{region}-{app}-{role}</code> | <code>us-east-ecom-api</code>   | Region, app, role           |
| <code>{team}-{service}-{id}</code> | <code>checkout-cart-a1b2</code> | Team, service, unique ID    |

## Host Naming via OneAgent

You can set a custom display name during installation:

```
sudo /bin/sh Dynatrace-OneAgent.sh --set-host-name="prod-web-01"
```

## Naming Principles

| Principle          | Good                       | Bad                        |
|--------------------|----------------------------|----------------------------|
| <b>Descriptive</b> | payment-service            | svc-001                    |
| <b>Consistent</b>  | prod-web-01 , prod-web-02  | prod-web-01 , Web Server 2 |
| <b>Parseable</b>   | us-east-prod-checkout      | USEastProdCheckout         |
| <b>Unique</b>      | Include environment/region | Generic names              |

## Property Naming Standards

For host properties, use consistent naming:

| Standard                 | Example                              | Why              |
|--------------------------|--------------------------------------|------------------|
| <b>Lowercase</b>         | env=prod not ENV=PROD                | Consistency      |
| <b>Hyphen separated</b>  | cost-center=eng                      | Readability      |
| <b>Short keys</b>        | env not environment                  | Query simplicity |
| <b>Consistent values</b> | Always prod not sometimes production | Filtering works  |

## 6. Querying by Tags and Properties

Use host properties and cloud tags in DQL queries to filter and group data.

```
// Find hosts by name pattern
fetch dt.entity.host
| filter contains(entity.name, "prod")
| fields entity.name
| limit 20

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST"
//   | filter contains(name, "prod")
//   | fields name
//   | limit 20
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
// Field maps: entity.name -> name.
```

```
// Count hosts by operating system type
fetch dt.entity.host
| summarize host_count = count(), by: {osType}
| sort host_count desc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST"
//   | summarize host_count = count(), by: {os.type}
//   | sort host_count desc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities
// than the classic entity store; for a pre-migration discovery inventory keep
the
// classic query above.
// Field maps: osType -> os.type (LINUX -> OS_TYPE_LINUX).
```

```
// Find services by name pattern
fetch dt.entity.service
| filter contains(entity.name, "checkout")
| fields entity.name, serviceType
| limit 20

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "SERVICE"
//   | filter contains(name, "checkout")
//   | fields name, dt.service.sdv1_type
//   | limit 20
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
// entities
// than the classic entity store; for a pre-migration discovery inventory keep
// the
// classic query above.
// Field maps: serviceType -> dt.service.sdv1_type; entity.name -> name.
```

```
// Query logs filtered by host group (Kubernetes)
fetch logs, from: now() - 1h
| filter isNotNull(k8s.namespace.name)
| summarize log_count = count(), by: {k8s.namespace.name}
| sort log_count desc
| limit 20
```

```
// Query spans by service name pattern
fetch spans, from: now() - 1h
| filter span.kind == "server"
| filter contains(service.name, "payment")
| summarize request_count = count(), by: {service.name}
| sort request_count desc
| limit 20
```

```
// Find hosts by name pattern for environment identification
fetch dt.entity.host
| filter not(contains(entity.name, "prod"))
      and not(contains(entity.name, "staging"))
      and not(contains(entity.name, "dev"))
| fields entity.name
| limit 20

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "HOST"
//   | filter not(contains(name, "prod"))
//           and not(contains(name, "staging"))
//           and not(contains(name, "dev"))
//   | fields name
//   | limit 20
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
// entities
// than the classic entity store; for a pre-migration discovery inventory keep
// the
// classic query above.
// Field maps: entity.name -> name.
```

## 7. Next Steps

---

With organization in place:

1. **ONBRD-07: Understanding Your Data** — Explore what Dynatrace discovered
2. Define host properties for your environment
3. Create segments for team-specific views
4. Document your naming conventions
5. Decide your `dt.security_context` value space (and bind IAM policies to it)

## Where to Go Deeper

- **ORGNZ series** (11 notebooks) — Bucket strategy, segments, security context, the full data-organization model
- **FAQ-01** — Host group naming strategy
- **FAQ-02** — Tagging sources, standards, and strategy (primary tags vs cloud tags vs auto-tags)

- **IAM series** (especially IAM-04, IAM-05, IAM-11 WORKSHOP) — Policy design that consumes `dt.security_context`
- **MZ2POL series** (11 notebooks) — Management Zone → Policy migration for legacy MZ retirement

## Organization Checklist

- [ ] Host property strategy documented
  - [ ] Properties + primary tags set at OneAgent install (sprint-1.337+ pattern, see ONBRD-05)
  - [ ] Cloud tags verified (if using cloud providers; see FAQ-02 for cloud-tag normalization)
  - [ ] `dt.security_context` value space decided
  - [ ] Segments created for common filters
  - [ ] Naming conventions established (FAQ-01 for host groups)
  - [ ] Access control configured via Policies (see ONBRD-02 + IAM series)
- 

## Summary

---

In this notebook, you learned:

- Why organization matters for scalability
  - The modern "tag at source" approach (primary fields + primary tags via OneAgent)
  - How to use host properties and cloud tags
  - How to create and use Segments for DQL-based filtering
  - Naming convention best practices
  - How `dt.security_context` standardizes the boundary field for Gen3 IAM
  - How to query by properties and attributes
- 

## References

---

- [Host Properties](#)

- [Segments](#)
  - [Cloud platform monitoring \(DT docs\)](#)
  - [Kubernetes Labels](#)
  - [DQL Reference](#)
  - [OneAgent Attribute Enrichment](#)
- 

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*